

AI large model environment deployment

The SSD mounting and Ollam model download path changes are optional for users.

AI large model environment deployment

1. Large Language Models (LLM)
2. Ollam Installation
 - Script Installation
3. Using Ollam
4. Uninstalling Ollama
5. Memory Allocation Adjustment
 1. Insufficient Available Memory at Factory Start
 2. Customers who need to adjust memory can continue reading
6. SSD Mounting (Optional)
 - The following are the mounting instructions for SSDs not included in the package:
7. Changing the Ollama Model Download Path (Optional)
8. Setting up the LargeModel Environment (Must Read!!!)
- References

Demo Environment

Development Board: RDK S600 Motherboard

Ollama is an open-source tool designed to simplify the deployment and operation of large language models, allowing users to use high-quality language models in their local environment.

1. Large Language Models (LLM)

Large Language Models (LLMs) are a class of advanced text generation systems based on artificial intelligence. Their main characteristic is their ability to learn and understand human language through large-scale training data and generate natural and fluent text.

2. Ollam Installation

Ollama is pre-installed in the YAHBOOM image.

Note: Ollam pull failures are normal. You can try multiple times, or try pulling using a proxy. You'll need to search online for instructions on using a proxy.

Script Installation

```
sudo apt install curl -y
sudo curl -fsSL https://ollama.com/install.sh | sh
```

```
sunrise@ubuntu: ~  
sunrise@ubuntu:~$ sudo apt install curl -y  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done  
curl is already the newest version (7.81.0-1ubuntu1.20).  
0 upgraded, 0 newly installed, 0 to remove and 243 not upgraded.  
sunrise@ubuntu:~$ sudo curl -fsSL https://ollama.com/install.sh | sh  
>>> Cleaning up old version at /usr/local/lib/ollama  
>>> Installing ollama to /usr/local  
>>> Downloading Linux arm64 bundle  
##### 17.2%
```

The entire installation process takes a relatively long time. Please be patient!

Once the installation is complete, a warning message will appear; ignore it:

```
sunrise@ubuntu:~$ sudo curl -fsSL https://ollama.com/install.sh | sh  
>>> Cleaning up old version at /usr/local/lib/ollama  
>>> Installing ollama to /usr/local  
>>> Downloading Linux arm64 bundle  
##### 100.0%  
>>> Creating ollama user...  
>>> Adding ollama user to render group...  
>>> Adding ollama user to video group...  
>>> Adding current user to ollama group...  
>>> Creating ollama systemd service...  
>>> Enabling and starting ollama service...  
Created symlink /etc/systemd/system/default.target.wants/ollama.service → /etc/s  
ystemd/system/ollama.service.  
>>> The Ollama API is now available at 127.0.0.1:11434.  
>>> Install complete. Run "ollama" from the command line.  
WARNING: No NVIDIA/AMD GPU detected. Ollama will run in CPU-only mode.  
sunrise@ubuntu:~$
```

Use `ollama -v` to check the Ollam version number:

```
sunrise@ubuntu:~$ ollama -v  
ollama version is 0.9.3
```

3. Using Ollam

Type `ollama` in the terminal to see usage prompts:

```
sunrise@ubuntu:~$ ollama
Usage:
  ollama [flags]
  ollama [command]

Available Commands:
  serve      Start ollama
  create     Create a model from a Modelfile
  show       Show information for a model
  run        Run a model
  stop       Stop a running model
  pull       Pull a model from a registry
  push       Push a model to a registry
  list       List models
  ps         List running models
  cp         Copy a model
  rm         Remove a model
  help       Help about any command

Flags:
  -h, --help      help for ollama
  -v, --version    Show version information

Use "ollama [command] --help" for more information about a command.
```

Command	Function
ollama serve	Start Ollam
ollama create	Create a model from a model file
ollama show	Display model information
ollama run	Run a model
ollama stop	Stop a running model
ollama pull	Pull a model from the registry
ollama push	Push a model to the registry
ollama list	List models
ollama ps	List running models
ollama cp	Copy a model
ollama rm	Delete a model
ollama help	Get help information for any command

4. Uninstalling Ollama

- Delete service

```
sudo systemctl stop ollama
sudo systemctl disable ollama
sudo rm /etc/systemd/system/ollama.service
```

- Delete file

```
sudo rm $(which ollama)
```

- Delete model and service user and group

```
sudo rm -r /usr/share/ollama
sudo userdel ollama
sudo groupdel ollama
```

5. Memory Allocation Adjustment

1. Insufficient Available Memory at Factory Start

Newly manufactured RDK S600 motherboards may have very little available memory. You can run the following two commands to update and increase the available memory:

```
sudo apt update
sudo apt upgrade
```

After updating, restart. You can then use the `free -h` command to see that your memory has increased.

2. Customers who need to adjust memory can continue reading

RDK provides a flexible memory allocation adjustment scheme.

This scheme supports multiple memory strategies and can be quickly switched according to the application scenario (such as BPU priority, CPU priority, or balanced mode).

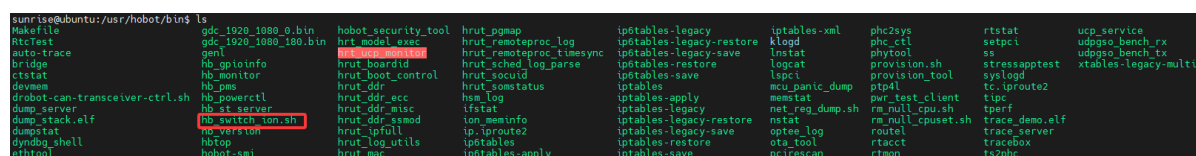
Since our factory-installed available memory is relatively small, some large models with large amounts of data cannot run, so we need to increase the available memory.

1. Open a terminal and enter the following commands:

```
cd /usr/hobot/bin/
```

Script Name: `hb_switch_ion.sh`

The script location is shown in the figure.



The script code is as follows:

```
#!/bin/bash

RED='\033[0;31m'
GREEN='\033[0;32m'
```

```

YELLOW='\033[0;33m'
NC='\033[0m'

log_error() {
    echo -e "${RED}[ERROR] $1${NC}" >&2
}

log_warn() {
    echo -e "${YELLOW}[WARN] $1${NC}" >&2
}

log_info() {
    echo -e "${GREEN}[INFO]${NC} $1"
}

help_msg()
{
    log_info "Usage: $0 <bpu_first | cpu_first | balanced | default>"
    log_info "Example: $0 bpu_first"
    exit 1
}

if [ $# -ne 1 ]; then
    help_msg
fi

target=$1

# Check if dtc command exists
if ! command -v fdtput &> /dev/null; then
    apt update
    apt install device-tree-compiler -y
fi

function get_dtb()
{
    boardid_sys_path="/sys/class/boardinfo/adc_boardid"
    if [ -f "$boardid_sys_path" ]; then
        boardid="$(cat $boardid_sys_path)"

        # get chip
        if [[ "$boardid" =~ ^0x(51)[1234567][01234567][1234567][1234567].$ ]];
then
            # S600
            echo "rdk-s600-mcb-v0p1.dtb"
        else
            # S100
            case $boardid in
                *"64"*) ;&
                *"65"*)
                    chip_str="s100p"
                    ;;
                *"6A"*) ;&
                *"6B"*)
                    chip_str="s100"
                    ;;
            esac
        fi
    fi
}

```

```

        case $boardid in
            *"60")
                hw_str="v1-21"
                ;;
            *"70")
                hw_str="v1-20"
                ;;
            *"84")
                hw_str="v0p5"
                ;;
            *"85")
                hw_str="v0p6"
                ;;
            *"86")
                hw_str="v1p0"
                ;;
        esac
        echo "rdk-${chip_str}-${hw_str}.dtb"
    fi
else
    echo "Invalid"
fi
}

dtb_name=$(get_dtb)

if [ "$dtb_name" = "Invalid" ];then
    log_error "Cannot find valid dtb for current board!"
    exit 1
fi

INPUT_DTB="/boot/hobot/${dtb_name}"
ORIG_INPUT_DTB="${INPUT_DTB}.bak"
OUTPUT_DTB="${INPUT_DTB}"

function reset_dtb()
{
    log_info "Restoring to default..."
    if [ -f "${ORIG_INPUT_DTB}" ];then
        cp "${ORIG_INPUT_DTB}" "${OUTPUT_DTB}"
        rm "${ORIG_INPUT_DTB}"
    fi
}

case $target in
    "balanced")
        if [[ "${INPUT_DTB}" = *-s600-* ]];then #total size 16GiB
            # ion-pool 2048MiB
            ion_pool_reg_val=(0x40 0xc0000000 0x0 0x80000000)
            # ion-carveout 10480MiB
            ion_carveout_reg_val=(0x41 0x40000000 0x2 0x80000000)
            # ion-cam 2048MiB
            ion_cma_reg_val=(0x43 0xc0000000 0x0 0x80000000)
            # ion-uncache 2048MiB
            ion_uncache_reg_val=(0x44 0x40000000 0x0 0x80000000)
        elif [[ "${INPUT_DTB}" = *-s100p-* ]];then
            # ion-pool 7680MiB
            ion_pool_reg_val=(0x4 0x00000000 0x1 0xe0000000)

```

```

        # ion-carveout 128MiB
        ion_carveout_reg_val=(0x8 0x00000000 0x0 0x08000000)
        # ion-cma 128MiB
        ion_cma_reg_val=(0xc 0x80000000 0x0 0x08000000)
    else
        # ion-pool 3840MiB
        ion_pool_reg_val=(0x4 0x00000000 0x0 0xF0000000)
        # ion-carveout 1280MiB
        ion_carveout_reg_val=(0x8 0x00000000 0x0 0x50000000)
        # ion-cma 1GiB
        ion_cma_reg_val=(0xc 0x80000000 0x0 0x40000000)
    fi
;;
"bpu_first")
    if [[ "${INPUT_DTB}" = *"-s600-"* ]];then #total size 28GiB
        # ion-pool 5120MiB
        ion_pool_reg_val=(0x40 0xc0000000 0x1 0x40000000)
        # ion-carveout 13312MiB
        ion_carveout_reg_val=(0x42 0x00000000 0x3 0x40000000)
        # ion-cma 5120MiB
        ion_cma_reg_val=(0x45 0x40000000 0x1 0x40000000)
        # ion-uncache 5120MiB
        ion_uncache_reg_val=(0x46 0x80000000 0x1 0x40000000)
    elif [[ "${INPUT_DTB}" = *"-s100p-"* ]];then
        # ion-pool 5120MiB
        ion_pool_reg_val=(0x4 0x00000000 0x1 0x40000000)
        # ion-carveout 5120MiB
        ion_carveout_reg_val=(0x8 0x00000000 0x1 0x40000000)
        # ion-cma 5120MiB
        ion_cma_reg_val=(0xc 0x80000000 0x1 0x40000000)
    else
        # ion-pool 3840MiB
        ion_pool_reg_val=(0x4 0x00000000 0x0 0xF0000000)
        # ion-carveout 3840MiB
        ion_carveout_reg_val=(0x8 0x00000000 0x0 0xF0000000)
        # ion-cma 1GiB
        ion_cma_reg_val=(0xc 0x80000000 0x0 0x40000000)
        log_warn "CAUTION! Setting bpu_first on S100 might introduce random
performance issue due to limited memory!"
    fi
;;
"cpu_first")
    if [[ "${INPUT_DTB}" = *"-s600-"* ]];then #total size 7GiB
        # ion-pool 2048MiB
        ion_pool_reg_val=(0x40 0xc0000000 0x0 0x80000000)
        # ion-carveout 2048MiB
        ion_carveout_reg_val=(0x41 0x40000000 0x0 0x80000000)
        # ion-cma 1024MiB
        ion_cma_reg_val=(0x41 0xc0000000 0x0 0x40000000)
        # ion-uncache 2048MiB
        ion_uncache_reg_val=(0x42 0x00000000 0x0 0x80000000)
    else
        # ion-pool 1GiB
        ion_pool_reg_val=(0x4 0x00000000 0x0 0x40000000)
        # ion-carveout 512MiB
        ion_carveout_reg_val=(0x8 0x00000000 0x0 0x20000000)
        # ion-cma 512MiB
        ion_cma_reg_val=(0xc 0x80000000 0x0 0x20000000)
    fi

```

```

        fi
        ;;
        "default")
            reset_dtb
            exit 0
        ;;
        *)
            log_error "Invalid option $target"
            help_msg
        ;;
    esac

# Set ion region sizes according to input

usage=$(df -h /boot | awk 'NR==2 {print $5}' | sed 's/%//')

if [ "$usage" -ge 95 ]; then
    log_error "/boot partition is full(${usage} used), cannot update dtb!"
    log_info "Maybe you should run the following command to resize the
filesystem:"
    log_info "    sudo resize2fs /dev/block/platform/by-name/boot_cur"
    exit 1
fi

if [ ! -f "${ORIG_INPUT_DTB}" ];then
    cp "${INPUT_DTB}" "${ORIG_INPUT_DTB}"
    log_info "Backup created:${ORIG_INPUT_DTB}"
else
    log_info "Backup(${ORIG_INPUT_DTB}) already exists, skip backup..."
fi

fdtput -t x "$INPUT_DTB" /reserved-memory/ion_reserved reg \
"${ion_pool_reg_val[@]}" || {
    log_error "Update ion-pool reg failed!"
    reset_dtb
    exit 1
}

fdtput -t x "$INPUT_DTB" /reserved-memory/ion_carveout reg \
"${ion_carveout_reg_val[@]}" || {
    log_error "Update ion_carveout reg failed!"
    reset_dtb
    exit 1;
}

fdtput -t x "$INPUT_DTB" /reserved-memory/ion_cma reg \
"${ion_cma_reg_val[@]}" || {
    log_error "Update ion_cma reg failed!"
    reset_dtb
    exit 1
}

if [[ "${INPUT_DTB}" = *-s600-* ]];then
    fdtput -t x "$INPUT_DTB" /reserved-memory/ion_uncache reg \
"${ion_uncache_reg_val[@]}" || {
        log_error "Update ion_uncache reg failed!"
        reset_dtb
        exit 1
    }
fi

```

```

    }
fi

log_info "ion_reserved reg = <$(fdtget -t x "$INPUT_DTB" /reserved-
memory/ion_reserved reg)>"
log_info "ion_carveout reg = <$(fdtget -t x "$INPUT_DTB" /reserved-
memory/ion_carveout reg)>"
log_info "ion_cma reg = <$(fdtget -t x "$INPUT_DTB" /reserved-memory/ion_cma
reg)>"
if [[ "${INPUT_DTB}" = *-s600-* ]];then
log_info "ion_uncache reg = <$(fdtget -t x "$INPUT_DTB" /reserved-
memory/ion_uncache reg)>"
fi

log_info "Update ${INPUT_DTB} for $target Done!"
log_info "The change will take effect AFTER reboot!"

```

2.Next, run the following in the terminal:

```
sudo ./hb_switch_ion.sh cpu_first
```

```

sunrise@ubuntu:/usr/hobot/bin$ sudo ./hb_switch_ion.sh cpu_first
[INFO] Backup created:/boot/hobot/rdk-s600-mcb-v0p1.dtb.bak
[INFO] ion_reserved reg = <40 c0000000 0 80000000>
[INFO] ion_carveout reg = <41 40000000 0 80000000>
[INFO] ion_cma reg = <41 c0000000 0 40000000>
[INFO] ion_uncache reg = <42 0 0 80000000>
[INFO] Update /boot/hobot/rdk-s600-mcb-v0p1.dtb for cpu_first Done!
[INFO] The change will take effect AFTER reboot!
sunrise@ubuntu:/usr/hobot/bin$

```

Finally, restart the motherboard, and then use the `free -h` command to see the changes in your memory allocation.

This script supports modification of the following four modes:

Mode Name	Description
bpu_first	Prioritizes memory allocation for BPUs, suitable for AI algorithm inference scenarios
cpu_first	Prioritizes memory allocation for CPUs, suitable for traditional computing load scenarios
balanced	Balances memory allocation between BPUs and CPUs, suitable for general scenarios
default	Restores factory default memory allocation (restores to .bak backup)

Our Ollama system relies entirely on CPU computation for running large models, so we use the `cpu_first` option.

6. SSD Mounting (Optional)

For UFS-booted systems only; skip this section if booting from NVMe.

Models pulled by Ollama are stored in UFS by default, consuming onboard storage space. You can expand storage with a solid-state drive and store large models on it. Simply follow the guide to change the model path used by Ollama or fastsdcpu to the SSD, so that existing models on the SSD can be accessed directly.

Ollama Model Specification: Skip to point 4 of [7. Changing the ollama Model Download Path \(Optional\)](#) in this tutorial and modify `ollama.service` to begin.

FastSD CPU Model Specification: Before starting the FastSD CPU web UI, enter `export HF_HOME=/media/ssd/HF_Model` and then start it. If you find it troublesome to type this every time you start the computer, you can directly add it to your `.bashrc` file.

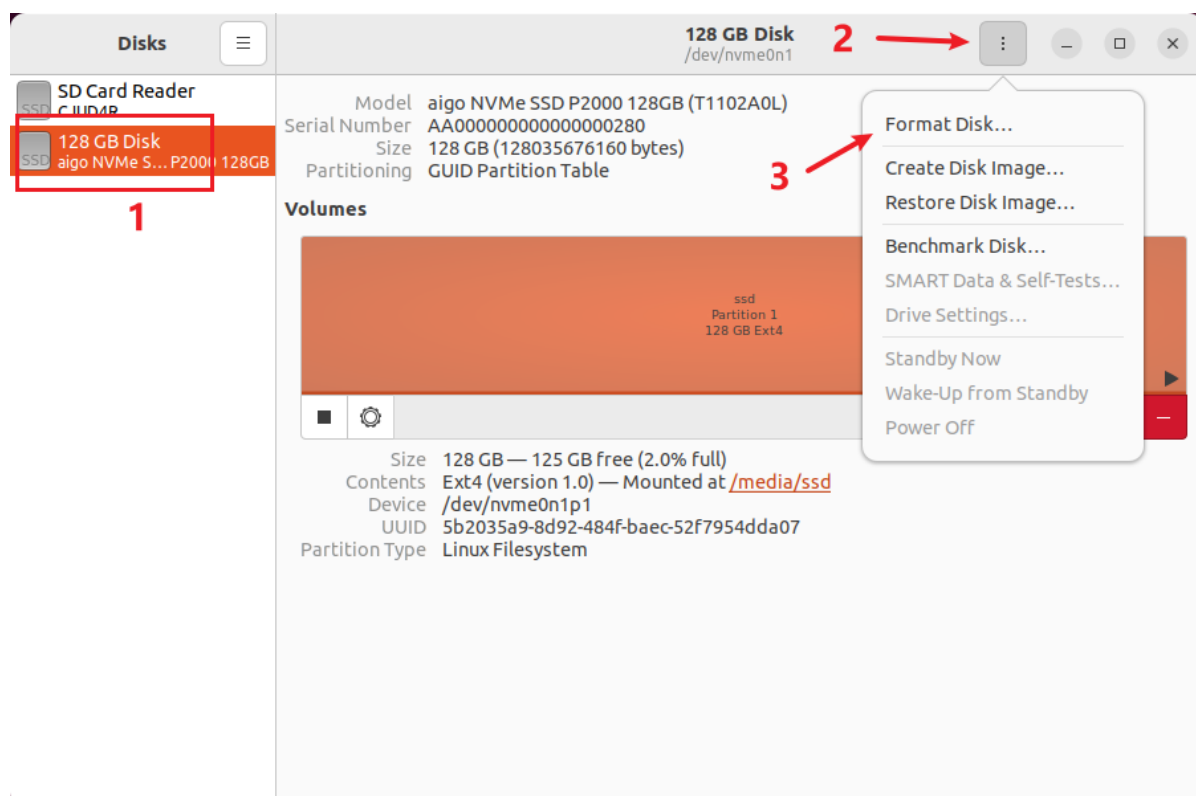
The default mount path for the SSD included in the package is: `/media/ssd`

The following are the mounting instructions for SSDs not included in the package:

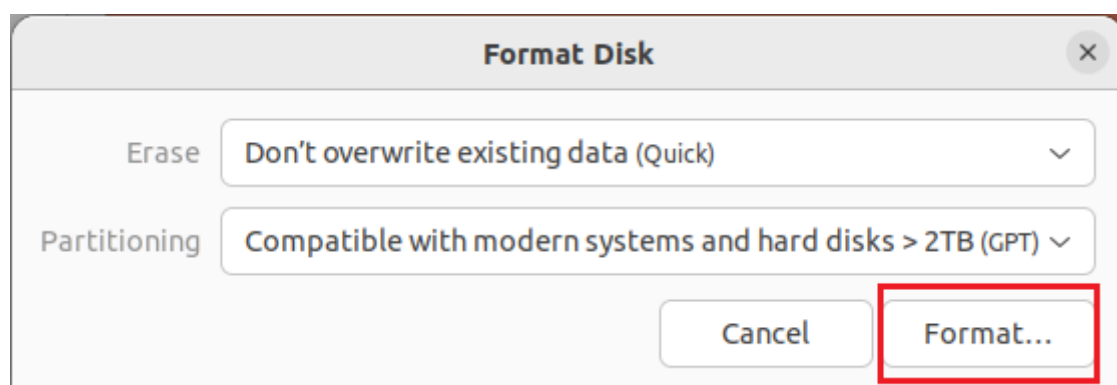
1. Open a terminal and enter the command to open Disks:

```
sudo gnome-disks
```

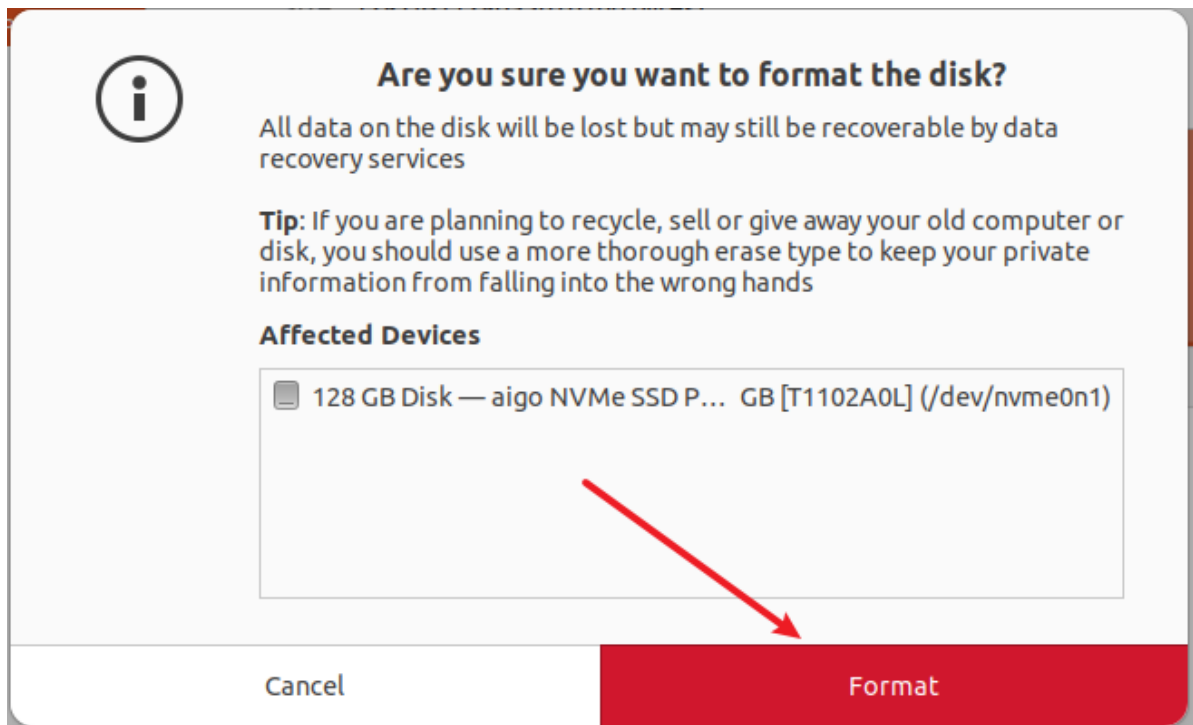
2. In the left sidebar of Disks, select the SSD you connected, and then click the three dots in the upper right corner of the interface. Then click Format Disk



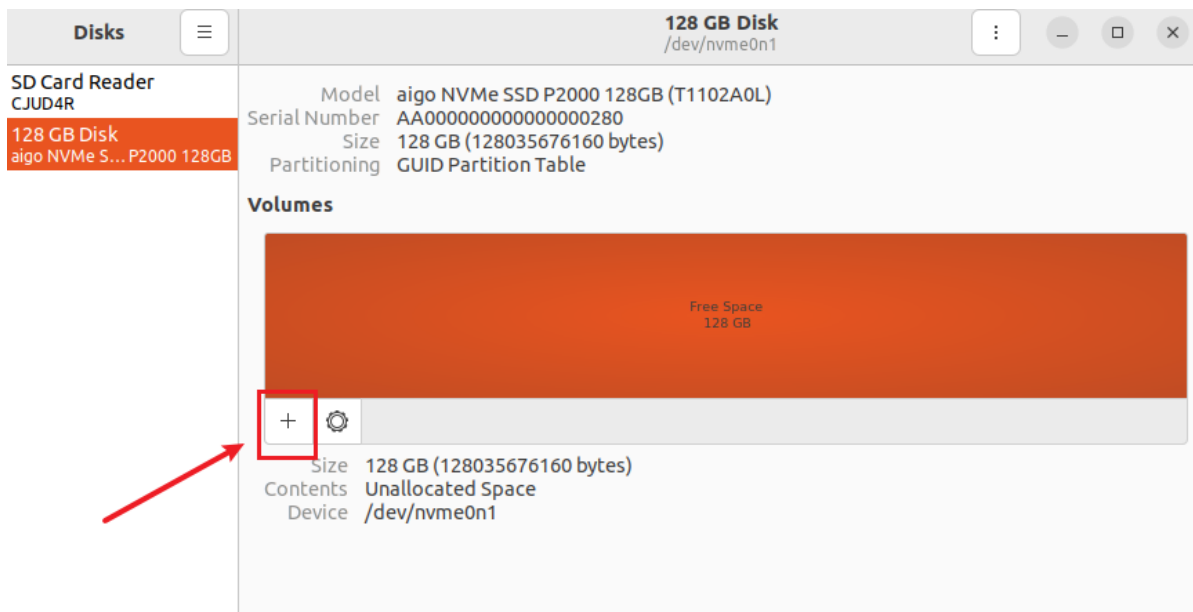
3. The default selection is fine; no changes are needed. Just click Format.



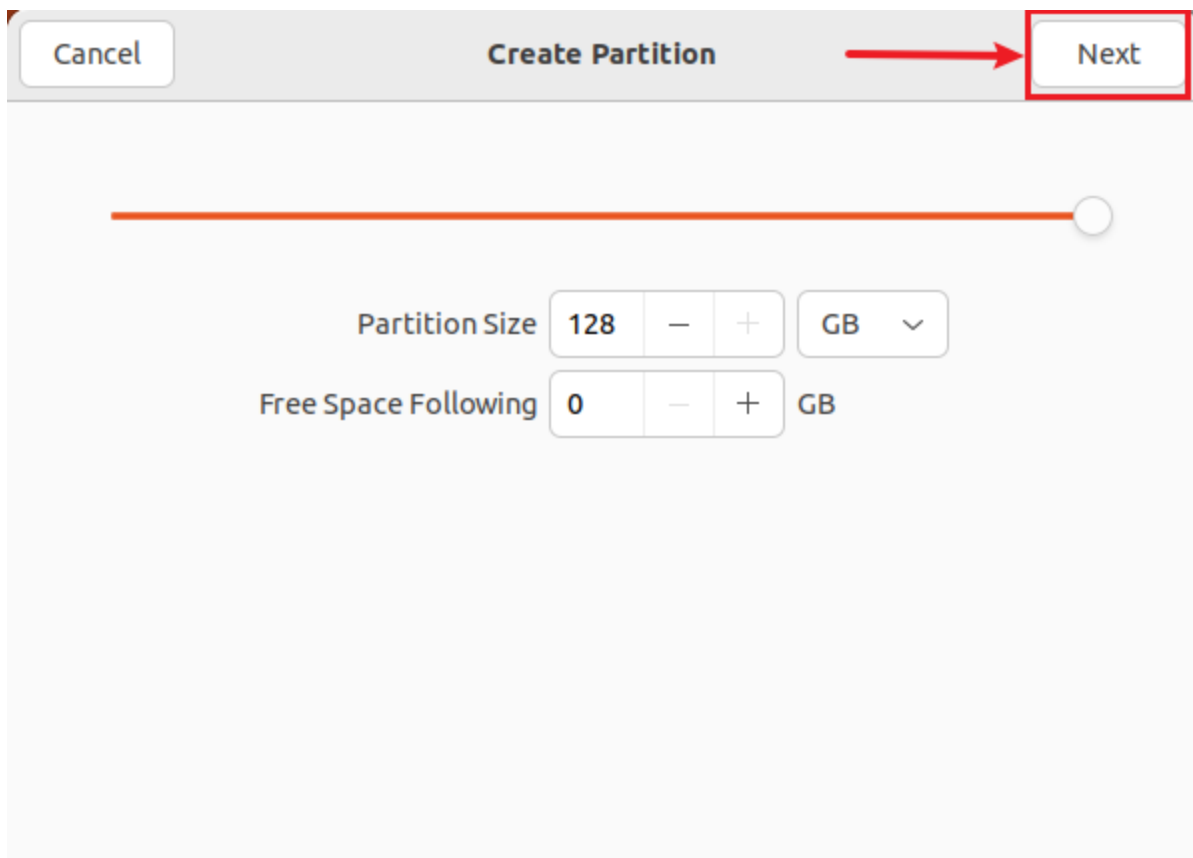
4. Click Format.



5. Click the plus sign to create a new partition.

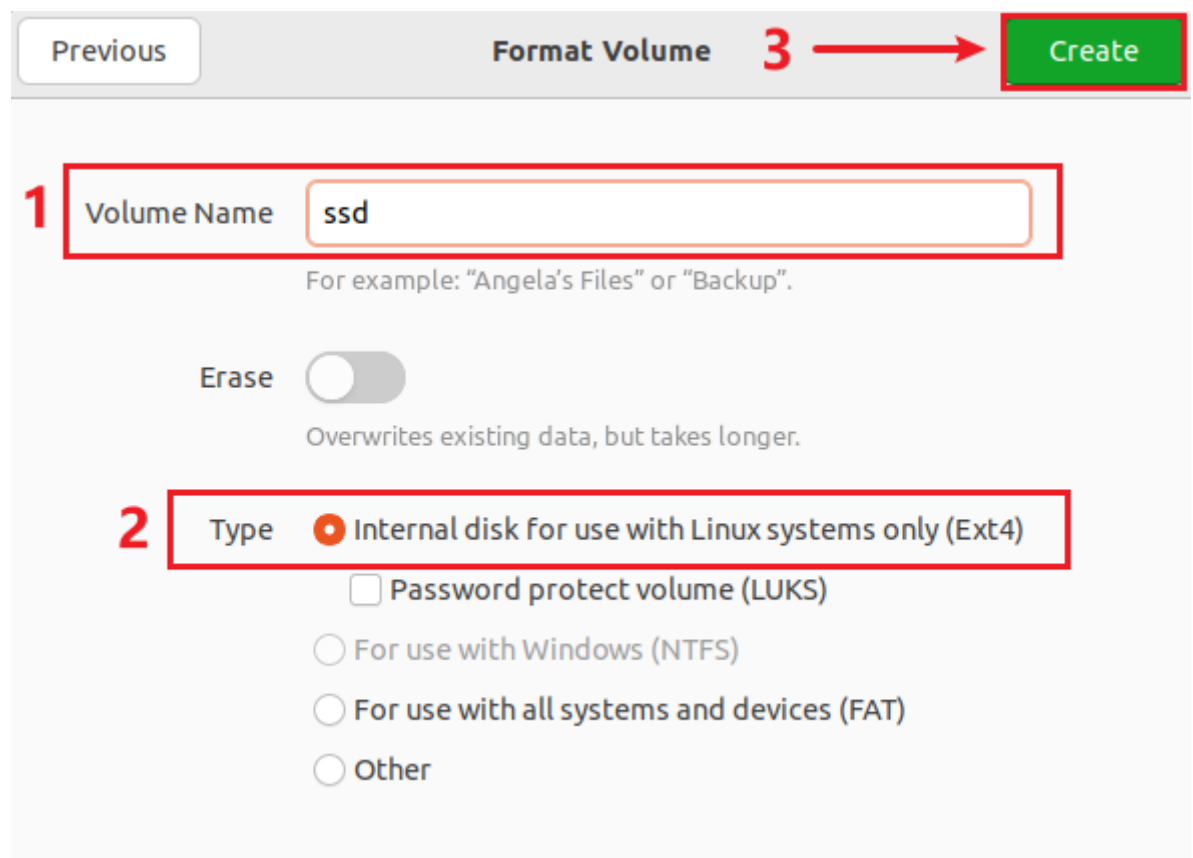


6. You can freely modify the partition size according to your needs. Here, I've defaulted to creating a 128GB partition. Click Next to proceed.



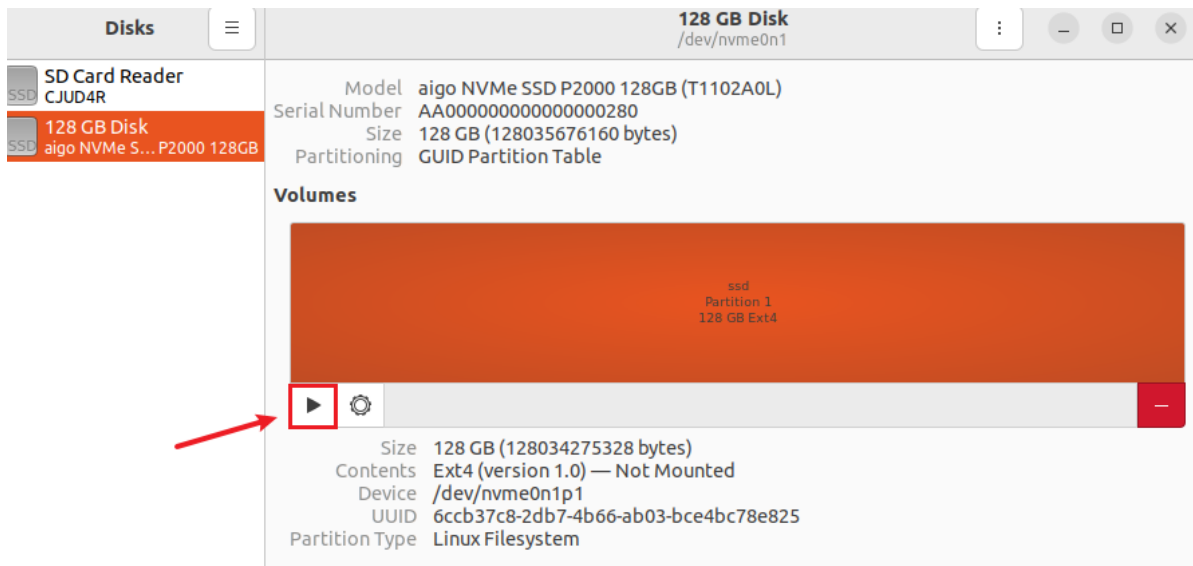
The 'Create Partition' dialog box features a top bar with 'Cancel', 'Create Partition', and 'Next' buttons. A red arrow points from the 'Create Partition' button to the 'Next' button. Below the bar is a progress slider. The main area contains two rows of controls: 'Partition Size' with a value of 128, minus and plus buttons, and a unit dropdown set to 'GB'; and 'Free Space Following' with a value of 0, minus and plus buttons, and a unit dropdown set to 'GB'.

7. Enter the disk name. The disk format must be Ext4; do not change it. Then click Create to create the partition.

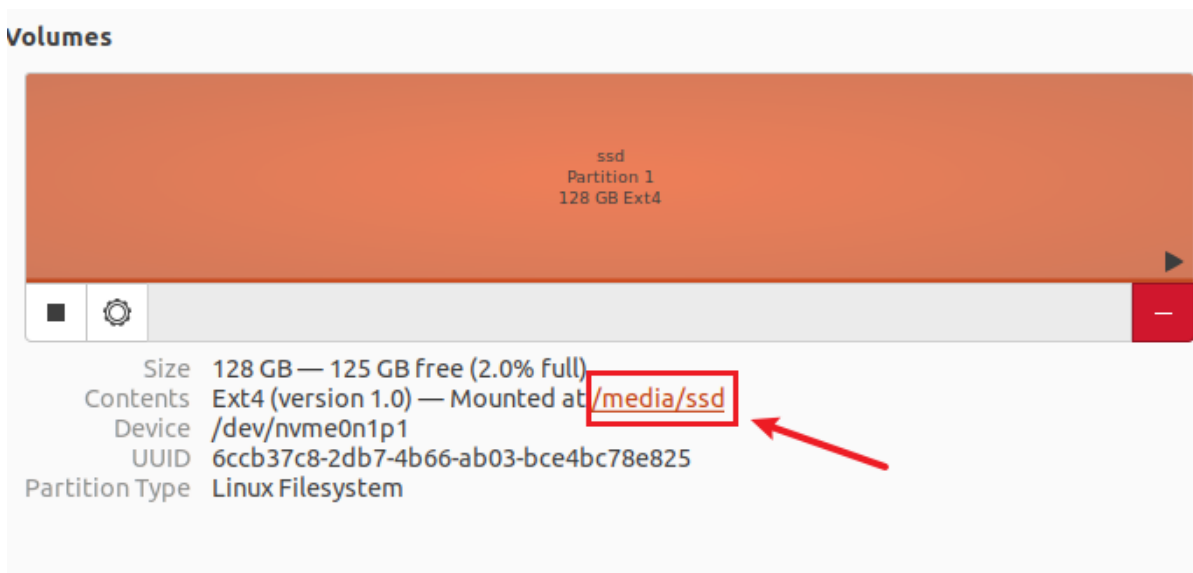


The 'Format Volume' dialog box has a top bar with 'Previous', 'Format Volume', and 'Create' buttons. A red '3' and a red arrow point from the 'Format Volume' button to the 'Create' button. The main area is divided into two sections. Section 1, labeled with a red '1', contains a 'Volume Name' text field with the value 'ssd', which is highlighted by a red box. Below it is the text 'For example: "Angela's Files" or "Backup".'. Section 2, labeled with a red '2', contains an 'Erase' toggle switch (currently off) with the text 'Overwrites existing data, but takes longer.' below it. Below the toggle is a red box containing the 'Type' section, which has a radio button selected for 'Internal disk for use with Linux systems only (Ext4)'. Other options include 'Password protect volume (LUKS)', 'For use with Windows (NTFS)', 'For use with all systems and devices (FAT)', and 'Other'.

8. After successful creation, click the play button to mount the SSD.

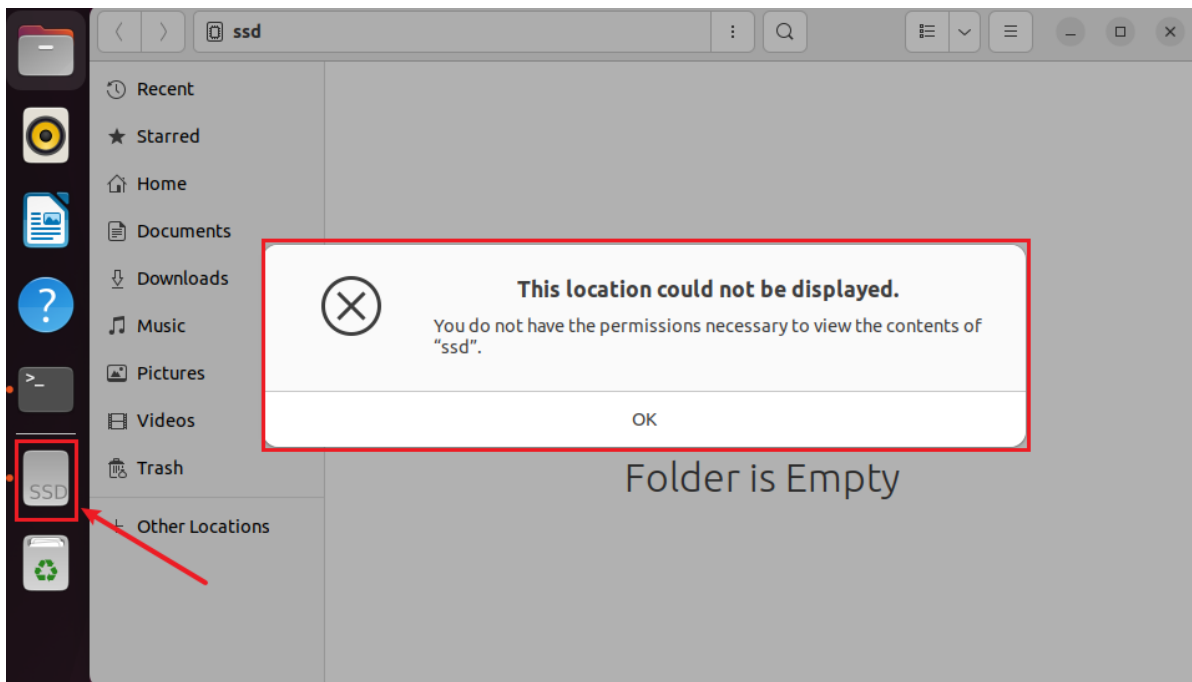


9. After successful mounting, you can see the SSD's mount path. In my case, it's located under /media/ssd.



10. Then we can see an SSD icon in the left sidebar of the desktop. We can click to enter the disk's folder, but we'll find that we don't have read or write permissions. At this point, we need to open a terminal and execute the command to use the disk normally:

```
sudo chmod 777 /media/ssd #Here, /media/ssd needs to be modified according to your SSD's mount path.
```



11. Finally, we need to set the hard drive to be automatically mounted at boot. First, enter the following command to query the SSD's UUID:

```
lsblk -f
```

```
sunrise@ubuntu: ~
|
|   ext4    1.0      e8dd09b7-ed6b-46a5-80dc-ff95d30c63ae    24.1M    48% /boot
|   └─mmcblk0p13
|   ext4    1.0      e8dd09b7-ed6b-46a5-80dc-ff95d30c63ae
|   └─mmcblk0p14
|   ext4    1.0      ota    9f5d7982-ca14-4b03-a3a7-6ef6dce7e4a1    7.4G     0% /ota
|   └─mmcblk0p15
|   ext4    1.0      log    ae58f475-5fc8-4b7b-a14b-76a4383b3e03    3.6G     0% /log
|   └─mmcblk0p16
|   ext4    1.0      userdata
|                               88602bfc-d79f-4bb0-9b12-1caca8f7c8b6    1.8G     0% /usr
|   rdata
|   └─mmcblk0p17
|   ext4    1.0      81c2ed79-a200-4534-97db-b90c40cb9d87    24.9G    40% /
|   mmcblk0boot0
|   mmcblk0boot1
|   nvme0n1
|   └─nvme0n1p1
|       ext4    1.0      ssd    6ccb37c8-2db7-4b66-ab03-bce4bc78e825    78.7G    27% /media/ssa/ssd
sunrise@ubuntu:~$
```

12. Open fstab

```
sudo vim /etc/fstab
```

Add the following text to the end, save and exit:

```
UUID=6ccb37c8-2db7-4b66-ab03-bce4bc78e825 /media/ssd ext4 defaults 0 2
#The content after UUID needs to be modified to your own value, which can be
found in the lsblk -f command above.
```

```
sunrise@ubuntu: ~  
# UNCONFIGURED FSTAB FOR BASE SYSTEM  
/dev/block/platform/by-name/boot_cur /boot ext4 defaults,noatime,nosuid,nodev,discards,noauto 0 1  
UUID=6ccb37c8-2db7-4b66-ab03-bce4bc78e825 /media/ssd ext4 defaults 0 2
```

7. Changing the Ollama Model Download Path (Optional)

Model download path modification applies only to systems booting from UFS.

On Linux systems, the default download path for Ollama models is:

```
/usr/share/ollama/.ollama/models
```

1. First, stop the Ollama service:

```
systemctl stop ollama.service
```

2. Create a new folder on your SSD to store the models using the following commands:

```
sudo mkdir -p /media/ssd/ollama/models # The /media/ssd here needs to be modified according to your SSD mount path
```

3. Then, add permissions to the model folder:

```
sudo chown -R sunrise:sunrise /media/ssd/ollama/models  
sudo chmod -R 777 /media/ssd/ollama/models
```

4. Next, we open ollama.service

```
sudo vim /etc/systemd/system/ollama.service
```

Add a new line under [Service]: Environment, then save and exit:

```
Environment="OLLAMA_MODELS=/media/ssd/ollama/models" # Remember to change the path!!!
```

```
sunrise@ubuntu: ~  
[Unit]  
Description=Ollama Service  
After=network-online.target  
  
[Service]  
ExecStart=/usr/local/bin/ollama serve  
User=ollama  
Group=ollama  
Restart=always  
RestartSec=3  
Environment="PATH=/usr/sbin:/usr/bin:/sbin:/bin:/usr/hobot/bin:/app/bin:/middleware/bin:/var/busybox:/usr/local/sbin:/usr/local/bin:/usr/games:/usr/local/games:/snap/bin"  
Environment="OLLAMA_MODELS=/media/ssd/ollama/models"  
  
[Install]  
WantedBy=default.target  
~  
~  
~  
~  
~  
12,52 All
```

5. Execute the following command to refresh the configuration:

```
sudo systemctl daemon-reload
```

6. Restart the ollama service and check its status:

```
sudo systemctl restart ollama.service  
sudo systemctl status ollama
```

```
sunrise@ubuntu:~$ sudo systemctl daemon-reload  
sunrise@ubuntu:~$ sudo systemctl restart ollama.service  
sunrise@ubuntu:~$ sudo systemctl status ollama  
● ollama.service - Ollama Service  
   Loaded: loaded (/etc/systemd/system/ollama.service; enabled; vendor preset: enabled)  
   Active: active (running) since Fri 2025-06-27 18:25:42 CST; 4s ago  
     Main PID: 8238 (ollama)  
       Tasks: 10 (limit: 3353)  
      Memory: 16.1M  
      CGroup: /system.slice/ollama.service  
              └─8238 /usr/local/bin/ollama serve  
  
Jun 27 18:25:42 ubuntu systemd[1]: Started Ollama Service.  
Jun 27 18:25:42 ubuntu ollama[8238]: time=2025-06-27T10:25:42.190Z level=INFO s>  
Jun 27 18:25:42 ubuntu ollama[8238]: time=2025-06-27T10:25:42.190Z level=INFO s>  
Jun 27 18:25:42 ubuntu ollama[8238]: time=2025-06-27T10:25:42.190Z level=INFO s>  
Jun 27 18:25:42 ubuntu ollama[8238]: time=2025-06-27T10:25:42.191Z level=INFO s>  
Jun 27 18:25:42 ubuntu ollama[8238]: time=2025-06-27T10:25:42.191Z level=INFO s>  
Jun 27 18:25:42 ubuntu ollama[8238]: time=2025-06-27T10:25:42.208Z level=INFO s>  
Jun 27 18:25:42 ubuntu ollama[8238]: time=2025-06-27T10:25:42.208Z level=INFO s>  
lines 1-17/17 (END)
```

At this point, the ollama model download path has been modified. New models pulled using ollama will be downloaded to the ollama/models folder on the SSD, reducing the space occupied by the eMMC.

8. Setting up the LargeModel Environment (Must Read!!!)

LargeModel is pre-installed in the YAHBOOM image.

The AI large-model multimodal applications in later chapters of this tutorial all require the LargeModel package to function properly.

1. Locate the largemodel.zip compressed file in the source code folder of your cloud drive. You can use a transfer tool like WinSCP to place the compressed file in your home directory, which is your ~ directory. Then run the command to decompress it:

```
unzip largemodel.zip
```

2. Run the installation script:

After decompression, first navigate to the LargeModel package:

```
cd ~/yahboom_ws/src/largemodel/
```

Then run the installation script:

```
chmod +x temp_install_script.sh #Add script execution permissions
./temp_install_script.sh
```

When running the script, it will ask if you are using a mainland China network. If so, enter 'Y' to switch to the Alibaba Cloud mirror source, which will speed up the environment download.

```
sunrise@ubuntu:~/yahboom_ws/src/largemodel$ ./installer
Initializing environment setup...

Are you in Mainland China and want to use a local mirror? (Y/n): y
-> Using a fast mirror for installation.

Starting environment installation. This may take a long time...
[sudo] password for sunrise:
[=====] 100% - Installing dependency 11/11...

=====
✅ All dependencies have been successfully installed!
✅ Environment is ready.
=====
```

3. Install the colcon tool

```
sudo apt update
sudo apt install python3-colcon-common-extensions
```

4. Add environment variables

Enter the following commands:

```
echo 'source /opt/ros/jazzy/setup.bash' >> ~/.bashrc
echo 'source ~/yahboom_ws/install/setup.bash' >> ~/.bashrc
```

The file `~/yahboom_ws/install/setup.bash` is generated only after running `colcon build`. You may ignore the file-not-found warning here.

Then enter:

```
source ~/.bashrc
```

5. Enter the workspace to compile

```
cd ~/yahboom_ws  
colcon build && source ~/yahboom_ws/install/setup.bash
```

6. Start the program:

```
ros2 launch largemodel largemodel_control.launch.py
```

This command is in voice mode and requires the AI large model voice module of the Yaboom RDK S600 to be connected. It detects the ttyUSB0 port number by default. For text input mode, please refer to subsequent tutorials.

References

Ollama

Official Website: <https://ollama.com/>

GitHub: <https://github.com/ollama/ollama>